Token-Backed Rolling the Dice
Project Report

COMP5521: Distributed Ledger Technology, Cryptocurrency and E-Payment

March 2026

1 Deployment Summary

- **Contract Addresses (Sepolia):**
  - **Token Contract (Stage1):** []
  - **Dice Contract (Stage2):** []
- **Tooling Used:** Remix IDE for compilation, deployment, and interaction; MetaMask for wallet management; Sepolia test network.
- **Betting/Reward Mechanism:** ETH-only betting with equal amounts from both players.
  - Winner-takes-all ETH plus a fixed token bonus.
  - 100 DICE tokens are paid per game from a pre-funded pool.
  - If the pool balance is below 100, the contract transfers the remaining balance instead.

2 Stage 1 Token: Design Notes

- **Internal state variables:** `_locked` (reentrancy guard), `owner` (contract owner, receives ETH on close), `_totalSupply` (total minted tokens), `_name` (token name), `_symbol` (token symbol), `_balances` (balances by address), and `PRICE_WEI_PER_TOKEN = 600` (fixed sale price).
- **Token acquisition path:** only `owner` can call `mint(address to, uint256 value)` to create new tokens and assign them to a specified address.
- **Sell mechanism:** users call `sell(uint256 value)` to sell tokens back to the contract at 600 wei/token. The contract checks ETH balance, burns the tokens by decrementing `_balances[msg.sender]` and `_totalSupply`, then sends ETH using `call{value: ...}` under `nonReentrant` protection.
- **Edge-case handling:**
  - Insufficient balance: `require(_balances[msg.sender] >= value)`.
  - Zero value: `require(value > 0, "cannot sell zero or below")` .
  - Insufficient ETH: `require(address(this).balance >= weiRequired)`.
  - Zero address: prevented in `transfer()` and `mint()`.

3 Stage 2 Dice Game: Protocol and State Machine

State Diagram

```
[None] -> [WaitingForB] -> [BothBetted] -> [Settled] -> [None]
   ^            v                v
    cancelGame()     checkTimeout()
```

State Descriptions

State	Allowed Actions	Who Can Call	Transitions
None	createDiceGame	Any player	→ WaitingForB
WaitingForB	joinGame, cancelGame	Any player (join), Creator only (cancel)	→ BothBetted, → None
BothBetted	revealA, revealB, checkTimeout	A (revealA), B (revealB), Anyone (timeout)	→ Settled
Settled	(game complete)	N/A	→ None (automatic reset)

Randomness Approach

- Mechanism: Commit-Reveal with multi-source entropy.
- Entropy sources: `secretA`, `secretB`, `block.number`, `block.timestamp`, `gamblerA`, `gamblerB`.
- Calculation: `keccak256(abi.encodePacked(secretA, secretB, block.number, block.timestamp, gamblerA, gamblerB))`.
- Winner rule: `(uint256(randomSeed) % 6) + 1` gives $n \in \{1, 2, 3, 4, 5, 6\}$; $n \in \{1, 2, 3\}$ means A wins, $n \in \{4, 5, 6\}$ means B wins.

Betting/Reward Mechanism

- What is bet: ETH only (equal amounts from both players).
- When deposits happen: Player A deposits in `createDiceGame()`, and Player B deposits in `joinGame()` with the same amount.
- Refund/forfeit conditions: `cancelGame()` lets the creator cancel within 30 minutes if nobody joined; `checkTimeout()` awards the revealed player if the other side fails to reveal after 30 minutes.
- Solvency guarantee: the winner receives all ETH held by the contract in `address(this).balance`; funds are only released at settlement or cancellation.
- Token bonus pool funding and size determination: Stage 1 tokens are pre-transferred to the Stage 2 contract; the bonus is fixed at 100 DICE per game, and the transfer falls back to the remaining pool balance when necessary.

4 Security & Fairness Design

Threat Model

Adversary	Description/Capability
Malicious Player A	May try to manipulate randomness or abort after learning the outcome.
Malicious Player B	May try to front-run or refuse to reveal.
Miner/Front-runner	May manipulate `block.timestamp` or reorder transactions.
Sybil Players	May try to play against themselves to drain the token pool.
Malicious Receiver Contract	May have a reverting fallback or attempt reentrancy.

Code-Level Hazards & Mitigations

Hazard	Mitigation
Reentrancy	`nonReentrant` on all ETH-transferring functions (`sell()`, `close()`, `_diceGameSettle()`, `checkTimeout()`, `cancelGame()`); state is updated before external calls.
DoS with revert	Checks-Effects-Interactions pattern; clear `require` messages; ETH is sent only after state updates.
State Machine Integrity	Enum-based state (`DiceGameState`) plus explicit state checks in every function; custom errors for invalid transitions.
Access Control	`require(msg.sender == owner)` for owner-only functions; `require(msg.sender == gamblerA/B)` for reveal functions.
Funds Safety	No permanent locks; `cancelGame()` and `checkTimeout()` ensure funds can always be recovered.
Input Validation	Zero-address checks, zero-value checks, and fingerprint validation.

Mechanism-Level Hazards & Mitigations

Hazard	Mitigation
Regret/Abort Prevention	30-minute timeout: if one player reveals and the other does not, the revealed player wins.
Randomness Manipulation	Multi-source entropy (6 sources) makes single-party manipulation infeasible; both players contribute secrets.
Front-Running	Commit-Reveal scheme: fingerprints are committed before reveal, preventing outcome prediction before commitment.
Token Pool Drain	Fixed bonus (100 DICE) limits per-game drain; equal bet requirement blocks micro-bet draining.
Sybil Attack	Equal ETH bets are required from both sides, so a Sybil attacker must risk real ETH to win the token bonus.
Denial of Service	`cancelGame()` lets the creator reclaim funds if no opponent joins within 30 minutes.
Double Settlement	The state machine checks `diceGameState == BothBetted` before settlement and immediately sets the state to `Settled`.
Replay Attacks	The game state resets after settlement, so old secrets and fingerprints cannot be reused.

Trade-offs

Trade-off Aspect	Decision & Justification
Security vs Gas Cost	Multi-source randomness and Commit-Reveal increase storage and computation, so gas is higher; this cost is accepted to preserve fairness. Fixed constants and efficient packing reduce overhead.
Simplicity vs Flexibility	A fixed 100 DICE bonus is simpler but less flexible than dynamic rewards; it was chosen for predictability and easier solvency verification.
Timeout Duration vs User Experience	The 30-minute timeout balances security (enough time for reveals) and usability (not waiting too long); it could be parameterized in future versions.
Push vs Pull Payment	Direct `call{value}` gives simple UX but can be blocked by a reverting receiver; a pull-payment design would be safer, but was not implemented due to added complexity and gas cost.

5 Gas and Fairness Evaluation

- **Deployment gas:** Stage 1 (Token) is approximately 800,000 gas; Stage 2 (Dice) is approximately 1,200,000 gas.

Operation	Estimated Gas	Who Pays
createDiceGame	~150,000	Player A
joinGame	~120,000	Player B
revealA	~80,000	Player A
revealB + settle	~180,000	Player B
Total	~530,000	Split: A ~230k, B ~300k

- **Fairness analysis:**
  - Player B pays slightly more gas because settlement happens in `revealB` if A revealed first.
  - This is acceptable because B has the information advantage of seeing A's commitment first.
  - No explicit gas reimbursement is implemented; gas is relatively small compared to bet amounts.
  - Both players benefit equally from winning.

6 Test Evidence

Full Game Execution on Sepolia

Step	Transaction Hash	Description
Create Game	`0x000000...0000`	Player A creates game with 0.01 ETH bet.
Join Game	`0x000000...0000`	Player B joins with matching 0.01 ETH bet.
Reveal A	`0x000000...0000`	Player A reveals secret.
Reveal B	`0x000000...0000`	Player B reveals secret and triggers settlement.

Token Pool Evidence

Event	Transaction Hash	Details
Pool Funding	`0x000000...0000`	10000 DICE transferred to Stage2.
Bonus Transfer	`0x000000...0000`	100 DICE sent to the winner.

Balance Verification

Address	Before Game	After Game
Winner ETH	X.XX ETH	X.XX + 0.02 ETH
Winner DICE	XXX DICE	XXX + 100 DICE
Stage2 ETH	0.02 ETH	0 ETH
Stage2 DICE	10000 DICE	9900 DICE

AI Usage Declaration

- **Tools Used:** Cursor; Gemini.
- **Scope of Use:** The team led all core work, including low-level algorithm design, optimization strategy, report structure, and main content writing. AI tools were used only as efficiency aids for architecture organization, Solidity syntax and gas optimization suggestions, debugging support, security/fairness review, and English polishing.
- **Verification:** The team reviewed, tested, and iterated on all AI-assisted code, and can explain and justify every implementation detail independently.

A Transaction History

Stage 1 Token Transactions

#	Operation	Transaction Hash	Notes
1	Deploy	0x000000...0000	Constructor: name="DiceToken", symbol="DICE"
2	Mint	0x000000...0000	Mint 1000 DICE to deployer
3	Transfer	0x000000...0000	Transfer 100 DICE to test address
4	Sell	0x000000...0000	Sell 50 DICE for 30000 wei

Stage 2 Dice Transactions

#	Operation	Transaction Hash	Notes
1	Deploy	0x000000...0000	Constructor: tokenContract=Stage1 address
2	Fund Pool	0x000000...0000	Transfer 10000 DICE to Stage2
3	Create Game	0x000000...0000	Player A, 0.01 ETH bet
4	Join Game	0x000000...0000	Player B, 0.01 ETH bet
5	Reveal A	0x000000...0000	Player A reveals secret
6	Reveal B	0x000000...0000	Player B reveals, settlement

B Source Code

Stage1.sol

Listing 1: Stage1.sol

```
pragma solidity ^0.8.0;

contract Stage1{
    // Reentrancy guard
    bool private _locked = false;

    modifier nonReentrant() {
        require(!_locked, "Reentrant call");
        _locked = true;
        _;
        _locked = false;
    }

    // State
    address payable public owner;

    // Events
    event Transfer(address indexed from, address indexed to, uint256 value);
    event Mint(address indexed to, uint256 value);
    event Sell(address indexed from, uint256 value);

    uint128 private constant PRICE_WEI_PER_TOKEN = 600;

    // Contract internal state
    uint256 private _totalSupply;
    string private _name;
```

```
    string private _symbol;
    mapping(address => uint256) private _balances;

    constructor(string memory name, string memory symbol) {
        owner = payable(msg.sender);
        _name = name;
        _symbol = symbol;
    }

    // View functions
    function getName() external view returns (string memory) {
        return _name;
    }

    function totalSupply() external view returns (uint256) {
        return _totalSupply;
    }

    function getSymbol() external view returns (string memory) {
        return _symbol;
    }

    function getPrice() external pure returns (uint128) {
        return PRICE_WEI_PER_TOKEN;
    }

    function balanceOf(address account) external view returns (uint256) {
        return _balances[account];
    }

    // State-Changing Functions
    function transfer(address to, uint256 value) external returns (bool) {
        require(to != address(0), "cannot transfer to 0x0");
        require(_balances[msg.sender] >= value, "no enough balance");

        _balances[msg.sender] -= value;
        _balances[to] += value;

        emit Transfer(msg.sender, to, value);
        return true;
    }

    function mint(address to, uint256 value) external returns (bool) {
        require(msg.sender == owner, "only owner may call");
        require(to != address(0), "mint to zero");

        _totalSupply += value;
        _balances[to] += value;
        emit Mint(to, value);

        return true;
    }

    function sell(uint256 value) external nonReentrant returns (bool) {
        require(value > 0, "cannot sell zero or below");
        require(_balances[msg.sender] >= value, "no enough balance");

        uint256 weiRequired = uint256(PRICE_WEI_PER_TOKEN) * value;
        require(address(this).balance >= weiRequired, "contract insufficient ETH");

        _balances[msg.sender] -= value;
        _totalSupply -= value;

        emit Sell(msg.sender, value);

        (bool sent, ) = payable(msg.sender).call{value: weiRequired}("");
        require(sent, "ETH transfer failed");
        return true;
    }

    function close() external {
        require(msg.sender == owner, "only owner may call");
        selfdestruct(owner);
    }

    receive() external payable {}
}
```

Stage2.sol

Listing 2: Stage2.sol

```
pragma solidity ^0.8.0;

interface IStage1 {
    function transfer(address to, uint256 value) external returns (bool);
    function balanceOf(address account) external view returns (uint256);
```

```
}

contract Stage2{
    bool private _locked = false;

    modifier nonReentrant() {
        require(!_locked, "Reentrant call");
        _locked = true;
        _;
        _locked = false;
    }

    enum DiceGameState { None, WaitingForB, BothBetted, Settled }
    DiceGameState public diceGameState;
    IStage1 public tokenContract;
    uint16 public constant TOKEN_BONUS = 100;
    uint256 public constant TIMEOUT_DURATION = 30 * 60;

    event DiceGameCreated(address gamblerA, uint256 betAmount, bytes32 fingerPrintForA);
    event DiceGameJoined(address gamblerB, uint256 betAmount, bytes32 fingerPrintForB);
    event BetForARevealed();
    event BetForBRevealed();
    event DiceGameSettled(address winner, uint256 profits, uint256 stage1TokenBonus);

    address public gamblerA;
    bytes32 public fingerPrintForA;
    bytes32 public secretA;
    bool public revealedA;

    uint256 public betAmount;

    address public gamblerB;
    bytes32 public fingerPrintForB;
    bytes32 public secretB;
    bool public revealedB;

    address public winner;

    uint256 public gameCreatedAt;
    uint256 public gameJoinedAt;

    error WrongState(DiceGameState expected, DiceGameState actual);
    error InvalidParam(string message);
    error NotBetOwner(address caller, address expected);
    error RepeatedRevealed(address caller);
    error BetMismatch(bytes32 expected, bytes32 actual);

    constructor(address tokenContractAddress){
        require(tokenContractAddress != address(0), "invalid tokenContractAddress");
        tokenContract = IStage1(tokenContractAddress);
    }

    function createDiceGame(bytes32 _fingerPrintForA) external payable nonReentrant{
        if (diceGameState != DiceGameState.None) revert WrongState(DiceGameState.None, diceGameState);
        if (msg.value == 0) revert InvalidParam("bet amount must be greater than 0");
        if (_fingerPrintForA == bytes32(0)) revert InvalidParam("fingerprint cannot be zero");

        gamblerA = msg.sender;
        betAmount = msg.value;
        fingerPrintForA = _fingerPrintForA;
        revealedA = false;
        gameCreatedAt = block.timestamp;
        diceGameState = DiceGameState.WaitingForB;
        emit DiceGameCreated(gamblerA, betAmount, fingerPrintForA);
    }

    function joinGame(bytes32 _fingerPrintForB) external payable nonReentrant {
        if (diceGameState != DiceGameState.WaitingForB) revert WrongState(DiceGameState.WaitingForB, diceGameState);
        if (msg.sender == gamblerA) revert InvalidParam("cannot play against yourself");
        if (msg.value != betAmount) revert InvalidParam("bet amount must match game bet amount");
        if (_fingerPrintForB == bytes32(0)) revert InvalidParam("fingerprint cannot be zero");

        gamblerB = msg.sender;
        fingerPrintForB = _fingerPrintForB;
        gameJoinedAt = block.timestamp;
        diceGameState = DiceGameState.BothBetted;
        emit DiceGameJoined(gamblerB, msg.value, fingerPrintForB);
    }

    function revealA(bytes32 _secretA) external nonReentrant {
        if (diceGameState != DiceGameState.BothBetted) revert WrongState(DiceGameState.BothBetted, diceGameState);
        if (msg.sender != gamblerA) revert NotBetOwner(msg.sender, gamblerA);
        if (revealedA) revert RepeatedRevealed(msg.sender);
        if (keccak256(abi.encodePacked(_secretA)) != fingerPrintForA) revert BetMismatch(fingerPrintForA, keccak256(abi.
     encodePacked(_secretA)));

        secretA = _secretA;
        revealedA = true;
        emit BetForARevealed();
        if (revealedB) _diceGameSettle();
```

```
}

function revealB(bytes32 _secretB) external nonReentrant {
    if (diceGameState != DiceGameState.BothBetted) revert WrongState(DiceGameState.BothBetted, diceGameState);
    if (msg.sender != gamblerB) revert NotBetOwner(msg.sender, gamblerB);
    if (revealedB) revert RepeatedRevealed(msg.sender);
    if (keccak256(abi.encodePacked(_secretB)) != fingerPrintForB) revert BetMismatch(fingerPrintForB, keccak256(abi.
 encodePacked(_secretB)));

    secretB = _secretB;
    revealedB = true;
    emit BetForBRevealed();
    if (revealedA) _diceGameSettle();
}

function _diceGameSettle() private nonReentrant{
    bytes32 randomSeed = keccak256(abi.encodePacked(
        secretA, secretB, block.number, block.timestamp, gamblerA, gamblerB
    ));
    uint256 n = (uint256(randomSeed) % 6) + 1;
    winner = n <= 3 ? gamblerA : gamblerB;
    uint256 profits = address(this).balance;
    uint256 balanceOfStage1 = tokenContract.balanceOf(address(this));
    uint256 stage1TokenBonus = balanceOfStage1 >= TOKEN_BONUS ? TOKEN_BONUS : balanceOfStage1;

    diceGameState = DiceGameState.Settled;
    emit DiceGameSettled(winner, profits, stage1TokenBonus);

    _reset();
    (bool sent, ) = payable(winner).call{value: profits}("");
    require(sent, "Bet profits failed to send");
    if (stage1TokenBonus > 0) {
        bool tokenSent = tokenContract.transfer(winner, stage1TokenBonus);
        require(tokenSent, "Stage1token bonus failed to send");
    }
}

function _reset() private {
    gamblerA = address(0);
    gamblerB = address(0);
    betAmount = 0;
    fingerPrintForA = bytes32(0);
    fingerPrintForB = bytes32(0);
    secretA = bytes32(0);
    secretB = bytes32(0);
    revealedA = false;
    revealedB = false;
    winner = address(0);
    gameCreatedAt = 0;
    gameJoinedAt = 0;
    diceGameState = DiceGameState.None;
}

receive() external payable {}

function checkTimeout() external nonReentrant {
    if (diceGameState != DiceGameState.BothBetted) revert WrongState(DiceGameState.BothBetted, diceGameState);
    if (block.timestamp <= gameJoinedAt + TIMEOUT_DURATION) revert InvalidParam("game has not timed out yet");

    if (revealedA && !revealedB) {
        winner = gamblerA;
    } else if (revealedB && !revealedA) {
        winner = gamblerB;
    } else {
        revert InvalidParam("invalid timeout state");
    }

    uint256 profits = address(this).balance;
    uint256 balanceOfStage1 = tokenContract.balanceOf(address(this));
    uint256 stage1TokenBonus = balanceOfStage1 >= TOKEN_BONUS ? TOKEN_BONUS : balanceOfStage1;

    diceGameState = DiceGameState.Settled;
    emit DiceGameSettled(winner, profits, stage1TokenBonus);

    _reset();
    (bool sent, ) = payable(winner).call{value: profits}("");
    require(sent, "Bet profits failed to send");
    if (stage1TokenBonus > 0) {
        bool tokenSent = tokenContract.transfer(winner, stage1TokenBonus);
        require(tokenSent, "Stage1token bonus failed to send");
    }
}

function getRemainingTimeout() external view returns (uint256) {
    if (diceGameState != DiceGameState.BothBetted) return 0;
    if (block.timestamp > gameJoinedAt + TIMEOUT_DURATION) return 0;
    return gameJoinedAt + TIMEOUT_DURATION - block.timestamp;
}

function cancelGame() external nonReentrant {
```

```
        if (diceGameState != DiceGameState.WaitingForB) revert WrongState(DiceGameState.WaitingForB, diceGameState);
        if (msg.sender != gamblerA) revert NotBetOwner(msg.sender, gamblerA);
        if (block.timestamp > gameCreatedAt + TIMEOUT_DURATION) revert InvalidParam("cancel period has expired");

        uint256 refundAmount = betAmount;
        _reset();

        (bool sent, ) = payable(msg.sender).call{value: refundAmount}("");
        require(sent, "Refund failed");
    }

    function getRemainingCancelTime() external view returns (uint256) {
        if (diceGameState != DiceGameState.WaitingForB) return 0;
        if (block.timestamp > gameCreatedAt + TIMEOUT_DURATION) return 0;
        return gameCreatedAt + TIMEOUT_DURATION - block.timestamp;
    }
}
```

Team Contribution Declaration

Member Information	**Member 1**	**Member 2**
Name	[Member 1 name missing]	[Member 2 name missing]
Student ID	[Member 1 student ID missing]	[Member 2 student ID missing]
Contribution Description	[Please add the exact contract/report work completed by Member 1]	[Please add the exact contract/report work completed by Member 2]
Percentage of Effort	[% missing]	[% missing]
Signature	[Signature missing]	[Signature missing]

By signing above, we declare that the work submitted is our own and that we have adhered to the academic integrity policies of the course.